

Zero-Trust Empowered Decentralized Security Defense against Poisoning Attacks in SL-IoT: Joint Distance-Accuracy Detection Approach

Rongxuan Song¹, Jun Wu^{1*}, Qianqian Pan^{2*}, Muhammad Imran³, Niddal Naser⁴, Rebet Jones⁵, Christos Verikoukis⁶

¹Graduate School of Information, Production and Systems, Waseda University, Japan

²Department of Computing and Mathematics, Manchester Metropolitan University, UK

³Institute of Innovation, Science and Sustainability, Federation University Australia

⁴College of Engineering, Alfaisal University, Riyadh, Saudi Arabia

⁵Department of Computer Science, Capitol Technology University, USA

⁶University of Patras and Iquadrat Information, Greece

*Corresponding author, Email: jun.wu@ieee.org, qianqian.pan@ieee.org

Abstract—Swarm learning (SL) exploits the blockchain to realize a federated and decentralized learning, which is very suitable for internet of things (IoT). Different from FL using central server to update global parameter, SL using edge node (header) to do that. However, poisoning attack is also an unresolved problem to SL. Because if header is malicious, it can pollute global parameter more easily than edge nodes. Moreover, there are following important limitations in existing defense schemes for FL, which cannot be used in SL directly. First, existing defense schemes focus on building a whitelist, which obstructs the decentralization because it can just provide decentralization in honest nodes instead of all of nodes. Second, existing schemes just consider poisoning attacks from edge nodes, they cannot defend attacks from header. Third, most existing schemes will let server execute the defense algorithm, but in SL, malicious header can return wrong defense results to deceive managers. To address above challenges, in this paper, we propose a protection system that leverages the concept of zero-trust architecture for SL, which achieves continuous risk calculation, analysis of learning behavior and abnormal parameter detection based on Manhattan distance and accuracy difference of parameters. We also evaluate the performance in the presence of random and customized malicious edge nodes. Experimental results demonstrate that our scheme can achieve higher accuracy than the other existing schemes.

Index Terms: Swarm Learning, Poisoning attack, Malicious user detection.

I. INTRODUCTION

Data islands and privacy leakage are two key challenges in the field of machine learning [1]. Thus, federated learning (FL) has become a very competitive technology in edge computing system by overcome above limitations [2] [3]. FL allows edge devices use their own data for machine learning with privacy protection. In FL, edge devices will train a local model by own data, then they just need to upload local model parameters to the server. Server will collect and aggregate them to update global parameter, then broadcast the new global parameter to edge devices for the learning of next round, and finally

gets a superior global model. However, using central sever for handling parameters concentrates power and decreases fault tolerance. So FL is not suitable for IoT because it is difficult to find a stable center in the IoT system, especially when edge devices are mobile phones or cars. On this basis, HPE proposed Swarm Learning (SL) architecture which uses the blockchain to achieve decentralized federated learning. It means that manager has no need to set a central server to update global parameter, all of the learning processes can be completed by edge devices [4]. Therefore, SL can achieve more stable and efficient federated learning in IoT.

However, FL is extremely easy to be attacked by data poisoning attack [5]. First, edge devices usually have poor security protection so attacker can easily obtain its authority and submit poisonous parameters to the server. Second, system will increase the number of edge devices as much as possible to get a better global model. There may be more than one malicious participant in the system. Third, server can only receive model parameters from edge devices for protecting privacy. Therefore, it is difficult for server to monitor whether the learning process of edge device is normal. Finally, local parameter may be quite different from each other that brings enormous difficulties for abnormal detection method.

What's worse, SL exhibits more inherent vulnerability to poisoning attack. As shown in Fig. 1, each learning node (ML node) has a SL Node for contacting other nodes. Whenever it is necessary to update global parameter, the system temporarily selects a SL Node as header. Header will collect local parameters from each node, then it will aggregate them and broadcast new global parameter to others. However, it should be noted that SL Node are also a part of edge node, which means header maybe malicious, it can pollute global parameter more easily and greatly.

While many defense schemes have been proposed to prevent data poisoning attack for FL, but they are not suitable for SL. For example, Ma et al. used the credibility to establish

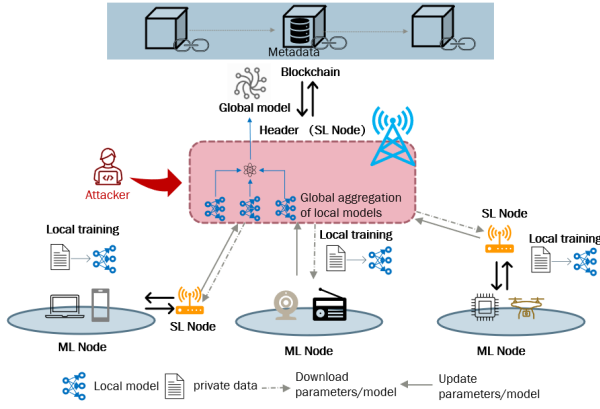


Fig. 1. Attacker can attack Header (SL node) in SL

a whitelist to manage nodes in FL, the server only accepts the parameter of trusted nodes to defend attackers [6]. But it can just provide decentralization in benign nodes instead of all nodes. Guo et al. let the server send a challenge to each edge node to test whether it is malicious, which can prevent attackers from uploading poisonous parameters [7]. However, this method is designed for centralized FL, which cannot defend attacks from header in SL. Moreover, these schemes assume that the server is honest, but in SL, malicious header can return incorrect challenge results to deceive managers.

Zero trust architecture (ZTA) that was proposed in [8] is very suitable for SL security, which keeps the basic assumption that any device inside or outside the network is untrustworthy before it is fully verified and authorized. However, existing zero-trust frameworks are not suitable for SL. First, many zero-trust frameworks mainly focus on how to design efficient access control strategies to reject illegal access, but in SL malicious header can legally access global parameter before launching poisoning attack. Second, existing zero-trust frameworks will reduce the authority of device without full verification to minimize the impact of attack. Although we can use multiple headers to jointly aggregate parameters, appropriate poisoning attack methods can still give significant damage to the model. Therefore, we need to adapt zero-trust architecture to SL that pay more attention to risk assessment and anomaly detection.

To this end, we designed a defense scheme base on the concept of ZTA for SL. Our main contributions of this article are summarized as follows:

- 1). We verified that there is a new security threat in SL, which can pollute global model more easily. The ZTA is a viable solution, but it needs to be changed for SL.
- 2). We proposed a defense scheme based on ZTA, which focus on risk calculation and analysis of learning behavior to defend poisoning attacks from both edge nodes and header in SL. To provide decentralization for most nodes, we will not allow only a small number of edge nodes can be selected as header. Each node has the probability of becoming the header based on their security level and the risk level of system.

3). We designed a detection algorithm based on Manhattan distance and accuracy difference of global parameters to prevent poisoning attacks from edge nodes. To prevent malicious header from returning incorrect detection results, system will select two edge nodes to complete the header's work together. One header aggregates local parameters, and the other tests global parameter and distribute it to other nodes. We also designed a extra-defense mechanism to ensure the correctness of the detection results, even though both headers are malicious.

The rest of this paper is organized as follows: Section 2 discusses relate works about defending poisoning attack. In Section 3 and 4, we present our scheme in detail. Section 5 analyzes performance of our scheme in multiple attack methods and datasets. We conclude this paper in Section 6.

II. RELATED WORK

There are many defense methods against poisoning attacks in FL now. Zhou et al. tested the norm distance and accuracy differences of each local parameter and average value of the other parameters to detect abnormal parameters [9]. Zhang et al. trained a confrontation model to judge which node is normal according to the logit of the data, he also conducted confrontation training to reduce the impact of data poisoning attacks [10]. Ma et al. proposed a strategy to defend data poisoning attacks on PPFL. It can use security cosine similarity of local parameters to detect if the encrypted parameters are safety. They also use Byzantine tolerance aggregation mechanism to ensure poisoning attack will not affect the accuracy [11]. Liu et al. designed a scheme based on isomorphic encryption technology, which allows the server to detect attackers through the effective gradient data extraction of the logarithmic function [12]. Huang et al. added isolated forest into FL to division the data space of parameters and calculate the probability of anomaly for each node. They also designed a threshold for reducing the impact of defense model to other nodes [13]. However, these methods are not suitable for SL. First, these methods can just provide protection against poisoning attacks from edge nodes. Second, to provide decentralization of whole system, defense method shouldn't allow only a few trusted nodes to be header. Last but not least, different from FL has honest server to detect abnormal parameter, header in SL may be malicious which can give wrong detection results to manager.

Distinguished with works mentioned above, in our scheme, there is another header for defending poisoning attack in aggregation process. Besides, risk calculation model and analysis of learning behavior algorithm can increase the possibility of selecting honest node as header while ensuring decentralization. Finally, extra-defense mechanism will restart conventional defense methods when detection results are unauthentic.

III. SCHEME OVERVIEW

A. System framework

Because the method designed in this article is mainly for preventing poisoning attacks in learning process. We build a

model that only simulates the learning part of SL by using flower [14]. The system includes ML node and SL node. There are 8 ML nodes in the system. The whole learning process is divided into two parts:

1). Pre-training: Every ML node will get randomly initialized parameters for training a few rounds. Which can get several global parameters to build the distance group.

2). Training: There are mainly the following steps:

First, each ML node uses local dataset for training to update local parameter. Second, system will select a SL node as header, since HPE does not give the code of aggregation part, we designed a basic selection method for testing model performance without defense scheme. Because header needs to collect local parameters, it can be considered that the header's performance is better than most of nodes in the network. Therefore, basic method will randomly select a part of nodes and evaluate their performance, the best one of them will be header. Third, each SL node sends local parameter to the header, header aggregates local parameters to get new global parameter. Last, header sends new global parameter to other SL nodes.

B. Assumptions

According to the system architecture we used in our experiments, we also have following assumptions. First, edge nodes can collect sufficient data to detect global parameters, because this part of data will be used to determine whether the global parameters have been poisoned. The scale of the dataset used for detection is not required to be large [15], which can be collected by users that have passed authentication or security verification. Second, the proportion of attackers in the network will remain constant, although system can remove malicious nodes out of the network, attackers can change device information to join the network again, so the system will always be in a dangerous environment.

C. Threat model

We build a model for simulating SL, which mainly faces poisoning attack from any nodes. There will be many malicious participants in the network for reducing the performance of global model. In existing works, attackers have two main ways to launch poisoning attack, data poisoning and model poisoning. Data poisoning signifies attackers will change the label of training dataset or add special data to reduce the learning effect of model. This attack method is more difficult to be detected. Model poisoning means that attackers will upload a randomly generated local parameters based on global parameter to affect global parameter. This attack method can bring greater impact to the model. If malicious nodes are not selected as header, they will send polluted parameters to the header. If header is malicious, it can pollute global parameter more easily and greatly which is shown by Fig. 2. If header is SA, it can increase the weight of poisoning parameter during aggregation to affect global model. If header is RA, it can broadcast a randomly generated parameter to other edge nodes instead of new global parameter.

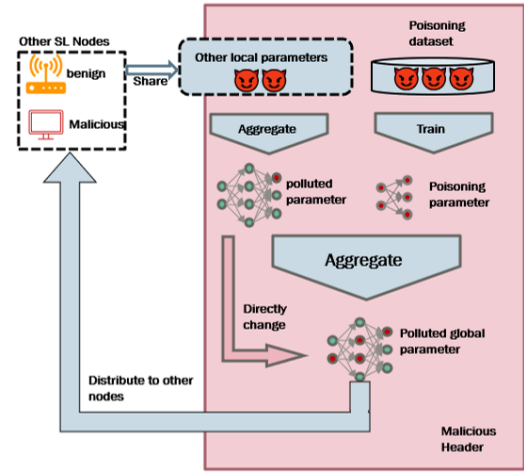


Fig. 2. Malicious header can pollute global parameter more easily than edge nodes.

D. Defense schemes

We designed three mechanisms to select benign node as header and defend poisoning attacks under zero-trust environment.

System will screened more credible edge nodes based on learning behaviors, and evaluated the risk of network based on the detection results. System will reduce the probability of nodes with higher score being selected as header to keep decentralization if risk level is low. Conversely, system will increase the probability of nodes with higher score being selected as header to protect global parameter. Besides, system does not completely trust the header. There are other methods in our scheme to prevent malicious header from destroying global model.

Header split: Two nodes jointly complete the header's task, one node is only responsible for collecting local parameters and aggregating them, then sends new global parameter to another header. Another header is responsible for testing new global parameter and distribution. If new parameter cannot pass the test, system will still use old global parameter for next training.

Extra defense mechanism: When there are too many attackers in the system, it is very possible that both headers are malicious. In this case, it is very hard for scheme to return correct detection result. Therefore, if system does not detect attacks for several consecutive iterations. system will randomly select two nodes from the nodes that are not selected as headers. Both will train a local parameter and use that to test whether global parameter is normal. If global parameter does not pass the test, it can be considered that there are too many malicious nodes in the network, defense scheme needs to be restarted to prevent subsequent attacks.

IV. DETAIL OF DEFENSE SCHEMES

At the beginning of the learning, system will evaluate the performance score PS_i of each device as their initial score. Each time a device completes the aggregation as the header,

its credit score CS_i will increase by $\beta/4$, other devices will increase by $\beta/10$. After that, whenever the global parameter need to be updated, the system will evaluate the score S_i of each node as $S_i = PS_i + \beta^{CS_i}$.

On this basis, system can calculate the risk level of this iteration as (1). Acc_k is the accuracy of global parameter in round k , and MS_k is the median accuracy of malicious node's S_i in round k .

$$R_k = Acc_{k-2}/4 + Acc_{k-1} - MS_k \quad (1)$$

If $R < MS_k/8$, it represents that attacker cannot significantly reduce the learning effect or there are a small number of attackers in the environment. The scheme will temporarily change each S_i as (2). T is the minimum of 0.05 and $(1 - S_i/HS_k)$. If $R > 0.8MS_k$, it means that attacker cannot significantly reduce the learning effect or there are many attackers in the environment. The scheme will temporarily change the S of lower nodes to zero.

$$S_i = S_i(1 - T) + \frac{\sum_{j=0}^N S_j T}{N} \quad (2)$$

At the last, the probability P_i of node i is selected as header will be calculated as (3).

$$P_i = \frac{S_i}{\sum_{j=0}^N S_j} \quad (3)$$

To ensure that header can be changed frequently, when the probability of a node being selected is higher than 75%, it will be deducted $\beta/2$ credit score to increase the opportunity of other nodes to be selected as header.

When global parameter needs to be updated, system will select aggregation header and detection header according to the score and risk level. The detection header mainly adopts the following methods to detect abnormal parameters.

1). Distance detection: In SL, each ML node will train their local model by own dataset. If learning process is normal, the whole model will gradually approach the best. Under the limit of learning rate, the difference between two parameters should be stable in a range. If the distance between new parameter and last parameter is too large, it can be suspected that this parameter is polluted. Here, in order to reduce the power of edge nodes, we use Manhattan distance, which is simple but still effective, to indicate the gap between local parameters.

2). Accuracy detection: distance detection can detect most of the abnormal parameters, but it cannot detect parameters which is trained by poisoning dataset, because these parameters are trained normally, it will just give error results in certain categories. Thus, we design the second detection scheme: accuracy detection. As we know, the data of edge nodes have high reliability, so we can use the data from edge nodes for accuracy testing to find the remaining abnormal parameters.

Based on the above two methods, we propose our parameter detection method to test whether the global parameter is poisoned. First, we pre-train for R rounds locally and get several model parameters $W_1, W_2, \dots, W_R$, then calculate the Manhattan distance between each two rounds of parameters

Algorithm 1: abnormal parameter detection

Input: Global parameter W_i and W_{i-1} , distance group $\{D_1, D_2, \dots, D_{R-1}\}$, variance benchmark η , accuracy benchmark κ , extend defense benchmark δ , Extra defense counter E_f

Output: whether W_i is poisoned

```

1 Compute the Manhattan distance between  $W_i$  and
   $W_{i-1}$ , then get new distance group
   $\{D_1, D_2, \dots, D_R\}$ 
2  $D_R = \sum_{j=1}^N |w_{ij} - w_{i-1j}|$ 
3 compute the variance of distance group and new group
4  $\sigma_1^2 = \frac{\sum_{i=1}^{R-1} (D_i - \bar{D})}{R-1}$ ,  $\sigma_2^2 = \frac{\sum_{i=1}^R (D_i - \bar{D})}{R}$ ,
   $\lambda = \sigma_2^2 - \sigma_1^2$ 
5 //distance detection
6 if  $\lambda \leq \eta$  then
7    $E_f - -$ 
8   return  $w_i$  is not poisoned
9 if  $\lambda \geq \eta + 1.5$  then
10   $E_f + +$ 
11  return  $w_i$  is poisoned
12 else
13   //accuracy detection
14   detection node uses  $D_{detect}$  get accuracy
    performance of global parameter  $W_i$  on each
    category  $A_i^1, A_i^2, \dots, A_i^n$ , then it will use  $W_{i-1}$  to
    get  $A_{i-1}^1, A_{i-1}^2, \dots, A_{i-1}^n$ 
15   $\Delta A_j = \frac{A_{i-1}^j - A_i^j}{A_{i-1}^j}$ 
16  if  $\sum_{j=1}^n \Delta A_j \geq \kappa$  then
17     $E_f + +$ 
18    return  $w_i$  is poisoned
19  else
20     $E_f - -$ 
21    return  $w_i$  is not poisoned
22 if  $E_f \leq -10$  then
23    $E_f = E_f + 5$ 
24   //Start the extend defense mechanism
25   two nodes use  $W_{local}$  and the previous global
    parameter get accuracy
     $\{A_{local}^1, A_{local}^2, \dots, A_{local}^n\}$  and
     $\{A_{global}^1, A_{global}^2, \dots, A_{global}^n\}$ 
26   $\Delta A_k = \frac{A_{local}^k - A_{global}^k}{A_{local}^k}$ 
27  if any  $\sum_{k=1}^n \Delta A_j \geq \delta$  then
28    return  $w_i$  is poisoned

```

to get the distance group $\{D_1, D_2, \dots, D_{R-1}\}$. Calculate the variance of distance group,

$$\sigma_1^2 = \frac{\sum_{i=1}^{R-1} (D_i - \bar{D})}{R-1} \quad (4)$$

When detection header gets new global parameter, it will calculate the Manhattan distance D_{n+1} between W_i and last

round global parameter W_{i-1} , add D_{n+1} to distance group and calculate the variance of new distance group,

$$\sigma_2^2 = \frac{\sum_{i=1}^R (D_i - \bar{D})}{R} \quad (5)$$

Define a detection benchmark η , let $\lambda = \sigma_2^2 - \sigma_1^2$, then if $\lambda \leq \eta$ this new parameter is normal so it can be used in next round of learning, delete the oldest Manhattan distance in the group. If $\lambda \geq \eta + 1.5$, it means that this parameter has been changed too much. It is likely that it has been poisoned. If $\eta \leq \lambda \leq \eta + 1.5$, detection node will use local dataset D_{detect} get accuracy performance of global parameter W_i on each category $A_1^1, A_1^2, \dots, A_1^n$, then it will use W_{i-1} to obtain the prediction accuracy of each category $A_{i-1}^1, A_{i-1}^2, \dots, A_{i-1}^n$, and finally we can get the difference of accuracy ΔA_j as (6).

$$\Delta A_j = \frac{A_{i-1}^j - A_i^j}{A_{i-1}^j} \quad (6)$$

Define another detection benchmark κ , if $\sum_{j=1}^n \Delta A_j \geq \kappa$, It can be considered that this parameter has been contaminated.

Extra defense mechanism: System will set a counter E_f at the beginning. Whenever the system detects abnormal parameter, the counter will be incremented one, otherwise it will be reduced one. When $E_f \leq -10$, system will randomly select two nodes from the outside of whitelist. Each node immediately uses local dataset D_{local} for training a round to get local parameter W_{local} , then uses W_{local} and the previous global parameter get accuracy $\{A_{local}^1, A_{local}^2, \dots, A_{local}^n\}$ and $\{A_{global}^1, A_{global}^2, \dots, A_{global}^n\}$, also we can get their difference ΔA_k as (7).

$$\Delta A_k = \frac{A_{local}^k - A_{global}^k}{A_{local}^k} \quad (7)$$

Define a detection benchmark δ , if any ΔA_k meets the conditions $\sum_{k=1}^n \Delta A_k \geq \delta$, it is possible that global parameter is poisoned so the model trained by ML nodes just one round can get much higher accuracy than it. It also can be considered that the whitelist has been occupied by attacker, and the conventional defense mechanism has no effect. In order to restart the defense mechanism, the system will randomly select a part of nodes outside the whitelist as the new whitelist. After that, system needs to train R rounds locally to rebuild a new distance group then counter will be incremented five times.

V. EXPERIMENTS

A. Experiment Environment

In this section, we will show the performance of our scheme on different attack methods and datasets. All the experiments were executed on a PC with an Intel Xeon Gold 6226R running at 2.90GHz processor and a NVIDIA RTX A5500.

Dataset: We use CIFAR-10, MNIST, for testing the performance of our scheme. MNIST consists of 60,000 training samples and 10,000 testing samples. CIFAR-10 includes 60,000 color images in 10 object classes. Every ML node will randomly get about 8% of training dataset for CIFAR10 learning, 3% for MNIST learning.

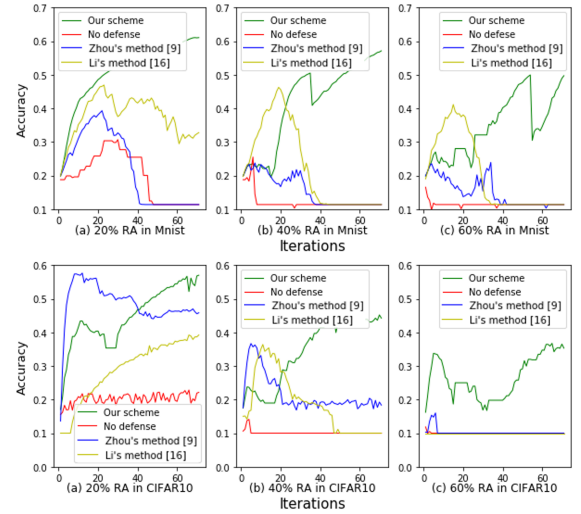


Fig. 3. The effect of our scheme under RA

Model: For MNIST, we use a three-layer network with 784-unit input layer, 128-unit hidden layer and 10-unit output layer. For CIFAR-10, we use the same model in PyTorch: A 60 Minute Blitz.

System Parameters: We set $\beta = 2$, $\eta = 0.3$, $\kappa = 0.5$, $\delta = 0.7$, $R = 5$, learning rate is 0.1 for MNIST, and 0.002 for CIFAR10.

B. Experiment Results

We execute experiments on two types of attackers to evaluate the defense effect of our method.

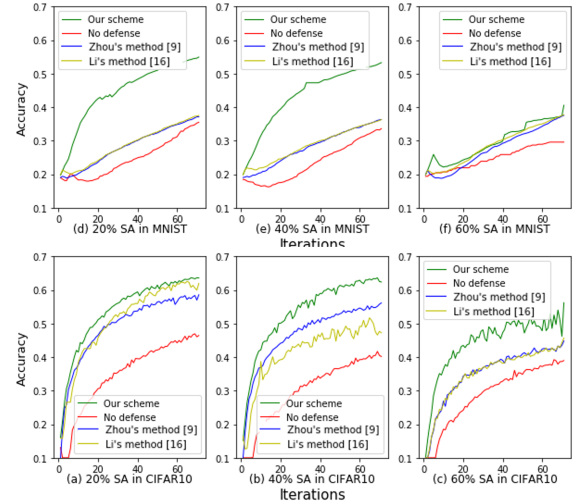


Fig. 4. The effect of our scheme under SA

The first type of attacker returns randomly generated local parameter to reduce the accuracy of model, which we call randomly attacker (RA). If header is RA, it will broadcast a random parameter, which has a significant impact on global and local parameters. If attacker modifies some labels of training dataset and trains a specialized parameter, which we call specialized aggregation attacker (SA). If header is SA,

it will increase the aggregation weight of own specialized parameter to enhance the impact of attacking.

1) Performance under poisoning attacks from any nodes:

We evaluate the effect of scheme on various attackers and datasets. We also reproduce the work of [9] and [16] and compare with our scheme in robustness against poisoning attacks. Fig. 3(a)-Fig. 3(c) shows that RA can cause significant damage to the model. The accuracy of several methods in the first 30 rounds can be higher than the model without defense scheme, but after that, only our method can maintain normal learning process, the accuracy of other methods is nearly 50% lower than ours. Because although other methods can defend attacks from edge nodes well, they cannot defend attacks from the header. That is also demonstrated from Fig. 3(d)-Fig. 3(f). According to the results, it can be seen that specialized attack do not significantly reduce the accuracy of the model, so they are more difficult to be detected. Our method can well defend malicious nodes and achieve about 40% higher accuracy than related schemes when there are fewer attackers, but sometimes when there are too many attackers in the network, our scheme can be deceived by malicious nodes. From the Fig. 4(a)-Fig. 4(f), it also can be seen that other schemes may get higher accuracy in the previous iterations of learning, but they cannot prevent malicious header from polluting model. Therefore, our scheme can reverse leading position in the later iterations. On the CIFAR10 dataset our scheme can protect model against attacks from many attackers, which can be specifically seen in Fig. 4(c). There are some short straight lines because our scheme prevents malicious header from polluting global parameter, so accuracy has not changed.

2) Performance under poisoning attacks from header:

We also test defense capability of scheme under attacks form header. According to Table. 1, it can be seen that our scheme can get about 18 higher accuracy even though there are 60% or 80% attackers in the system.

TABLE I

THE COMPARISON RESULT OF ACCURACY UNDER ATTACKS FROM HEADER

Dataset	Scheme	The proportion of attackers			
		Specialized Attacker		Random Attacker	
		60%	80%	60%	80%
MNIST	Ours	45.3	31.2	47.4	25.5
	No defense	25.7	13.5	10	10
CIFAR10	Ours	52.8	45.6	37.9	33.2
	No defense	42.7	36.1	10	10

VI. CONCLUSION

In this paper, a differential scheme against poisoning attacks for SL was proposed. The scheme is designed to defend poisoning attacks in aggregation and provide the decentralization for most nodes. Specifically, it adapts zero-trust architecture for SL, which achieves continuous risk calculation, analysis of learning behavior to reduce the probability of selecting malicious nodes as header. It can also detect abnormal parameter based on Manhattan distance and accuracy difference of parameters. Experiment results show that our scheme has

higher accuracy in most case. In future work, we will test time cost and add privacy protecting in this scheme. We will also improve the performance of detection algorithm in the environment of many attackers.

ACKNOWLEDGMENT

This work was supported in part by the JSPS KAKENHI under Grants 23K11072, in part by the National Natural Science Foundation of China under Grants U21B2019 and 61972255, in part by the EC funded project 5GMediaHUB (101016714) under H2020.

REFERENCES

- [1] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan and X. Chen, "Convergence of Edge Computing and Deep Learning: A Comprehensive Survey," in *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869-904, Secondquarter 2020.
- [2] Q. Pan, J. Wu, A. K. Bashir, J. Li, W. Yang, Y. D. Al-Otaibi, "Joint Protection of Energy Security and Information Privacy for Energy Harvesting: An Incentive Federated Learning Approach," in *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3473-3483, 2022.
- [3] J. Zhu, J. Wu, A. K. Bashir, Q. Pan, W. Yang, "Privacy-Preserving Federated Learning of Remote Sensing Image Classification With Dishonest Majority," in *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 4685-4698, 2023.
- [4] Warnat-Herresthal, S., Schultze, H., Shastry, K.L. et al. Swarm Learning for decentralized and confidential clinical machine learning, in *Nature* 594, 265-270 (2021).
- [5] J. Zhang, B. Chen, X. Cheng, H. T. T. Binh and S. Yu, "PoisonGAN: Generative Poisoning Attacks Against Federated Learning in Edge Computing Systems," in *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3310-3322, 1 March 1, 2021, doi: 10.1109/JIOT.2020.3023126.
- [6] X. Ma, Q. Jiang, M. Shojafar, M. Alazab, S. Kumar and S. Kumari, "DisBeZant: Secure and Robust Federated Learning Against Byzantine Attack in IoT-Enabled MTS," in *IEEE Transactions on Intelligent Transportation Systems*, doi: 10.1109/TITS.2022.3152156.
- [7] J. Guo et al., "ADFL: A Poisoning Attack Defense Framework for Horizontal Federated Learning," in *IEEE Transactions on Industrial Informatics*, vol. 18, no. 10, pp. 6526-6536, Oct. 2022, doi: 10.1109/TII.2022.3156645.
- [8] E. Gilman and D. Barth, "Zero Trust Networks: Building Secure Systems in Untrusted Networks," Sebastopol, CA, USA: O'Reilly Media, 2017.
- [9] J. Zhou et al., "A Differentially Private Federated Learning Model against Poisoning Attacks in Edge Computing," in *IEEE Transactions on Dependable and Secure Computing*, doi: 10.1109/TDSC.2022.3168556.
- [10] J. Zhang, C. Ge, F. Hu and B. Chen, "RobustFL: Robust Federated Learning Against Poisoning Attacks in Industrial IoT Systems," in *IEEE Transactions on Industrial Informatics*, vol. 18, no. 9, pp. 6388-6397, Sept. 2022, doi: 10.1109/TII.2021.3132954.
- [11] Z. Ma, J. Ma, Y. Miao, Y. Li and R. H. Deng, "ShieldFL: Mitigating Model Poisoning Attacks in Privacy-Preserving Federated Learning," in *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639-1654, 2022, doi: 10.1109/TIFS.2022.3169918.
- [12] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang and R. Lu, "Privacy-Enhanced Federated Learning Against Poisoning Adversaries," in *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4574-4588, 2021, doi: 10.1109/TIFS.2021.3108434.
- [13] S. Huang, Y. Bai, Z. Wang and P. Liu, "Defending against Poisoning Attack in Federated Learning Using Isolated Forest," 2022 2nd International Conference on Computer, Control and Robotics (ICCCR), 2022, pp. 224-229, doi: 10.1109/ICCCR54399.2022.9790094.
- [14] Beutel, et al. "Flower: A Friendly Federated Learning Research Framework," 2020.
- [15] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y. C. Liang, Q. Yang, D. Niyato, and C. Miao, "Federated learning in mobile edge networks: A comprehensive survey," in *IEEE Communications Surveys Tutorials*, vol. 22, no. 3, pp. 2031-2063, 2020.
- [16] X. Li, et al., "LoMar: A Local Defense Against Poisoning Attack on Federated Learning" in *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 01, pp. 437-450, 2023.